

For this lab, I implemented two procedural audio patches in Python. The first was a babbling brook, based on a short SuperCollider program provided in the assignment prompt. The second was a thunderstorm ambience, which I selected as my more challenging “extra step” sound.

The goal of the brook patch was to recreate the bubbling, watery texture of the SuperCollider example:

I interpreted this patch as using two streams of brown noise. The first stream serves as the primary sound source and is low-pass filtered at 400 Hz to remove high-frequency content and create a smoother texture. The second stream is also brown noise, but it is filtered at 14 Hz, causing it to change very slowly over time. This slow signal functions as a control signal rather than an audible sound.

This control signal is then scaled and used to modulate the cutoff frequency of a resonant high-pass filter. As the cutoff frequency shifts over time, different parts of the noise spectrum are emphasized, producing a dynamic, bubbling effect that resembles flowing water.

In Python, I approximated the resonant high-pass filter using a time-varying biquad filter. This was the main technical challenge in this part, since Python does not provide a direct equivalent to SuperCollider’s RHPF unit generator. Implementing this required manually updating filter parameters over time to simulate resonance and movement.

For the open-ended portion of the lab, I chose to build a thunderstorm ambience. This was a more complex and challenging sound because it required combining multiple synthesis techniques rather than relying on a single signal chain.

The thunderstorm consists of four main components:

- **Rain layer:** White noise filtered through a high-pass and low-pass filter to create a soft, continuous spray-like texture.
- **Wind layer:** Brown noise filtered to emphasize low frequencies, with slow amplitude variation to simulate natural gusts.
- **Thunder events:** Short bursts of low-frequency noise shaped by an amplitude envelope to create rumbling impacts.
- **Echo and space:** A delay and feedback system to simulate distance and environmental reflections, making the thunder feel more immersive.

This patch combines several synthesis techniques, including noise-based synthesis, subtractive synthesis, modulation, and time-based effects. I chose noise as the primary sound source because environmental sounds like rain, wind, and thunder are inherently non-harmonic. Filtering and envelope shaping were then used to sculpt this noise into recognizable sound events.

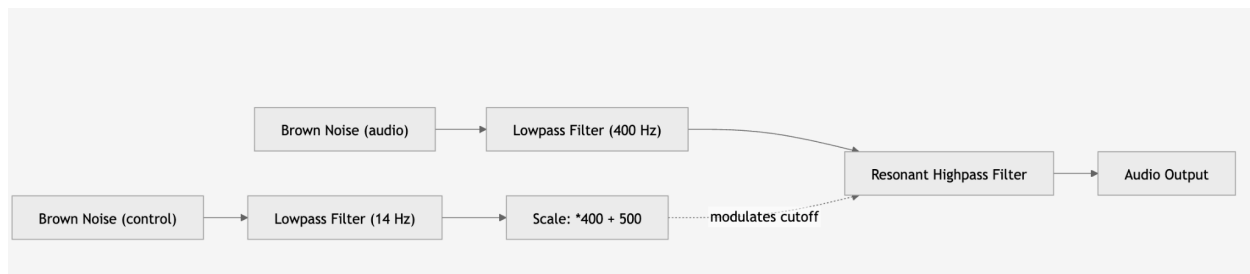
This lab was more open-ended than previous assignments, which initially made it feel less structured. However, it became more manageable once I began thinking in terms of layers rather than trying to directly replicate real-world sounds.

Instead of aiming for perfect realism, I focused on identifying the key components that make a sound recognizable. For the thunderstorm, layering several simple elements produced a much richer and more convincing result than trying to design a single complex system.

The babbling brook portion helped me practice reading unfamiliar audio code and translating it into a different environment. The thunderstorm pushed me to make more independent design decisions and experiment with combining different synthesis techniques.

While the final sounds are not perfect physical simulations, they successfully demonstrate how procedural audio can be used to create believable soundscapes from relatively simple building blocks

Brook Signal Flow



Thunderstorm Signal Flow

